Expressões *lambda*

Bib: https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html

Programação III (PG3)

João Pedro Patriarca, (joao.patriarca@isel.pt),

Gabinete F.0.23 do edifício F, LEETC, ADEETC



Resumo

- Caso de estudo: apresentar, com base num critério de seleção, informação de pessoas. Os objetos do tipo `Person` estão presentes numa coleção

- Expressões *lambda*

- Referência a métodos

Resumo

- Caso de estudo: apresentar, com base num critério de seleção, informação de pessoas. Os objetos do tipo `Person` estão presentes numa coleção
  1. Iteração externa
  2. Parâmetro com critério de seleção
  3. Parâmetro com classe anónima
  4. Parâmetro com expressão *lambda*
  5. Parâmetro com `Predicate`
  6. Parâmetro com `Consumer`
  7. Parâmetro com `Function`
  8. Método genérico
  9. Referências para métodos
  10. Operações agregadas

Genericamente faz:

1. Itera (sobre cada elemento da coleção)
2. Filtra (elementos da coleção)
3. Aplica ação (a cada elemento que passa no filtro)

Classe Person

```
public class Person {
   public static final String NO_EMAIL = "no email address";
   public enum Gender {MALE, FEMALE}
   private final String name;  private final LocalDate birthday;
   private final Gender gender; private String email;

   public Person(String n, Gender g, LocalDate b, String e) { name = n; gender = g; birthday = b; email = e;  }
   public int getGender() { return gender; }
   public int getAge() { return Period.between(birthday, LocalDate.now()).getYears(); }
   @Override public String toString() {
      return name + " [" + gender + '|' + birthday + "]: " + (email == null ? NO_EMAIL : email);
   }
   public void printPerson() { System.out.println(this); }
}
```

1. Iteração externa

```
public class PersonDemo {
    public static void printWithinAgeRangeDemo(List<Person> roster) {
        for (Person p : roster) {
            if (p.getGender() == Person.Gender.MALE && 18 <= p.getAge() && p.getAge() <= 25) {
                p.printPerson();
            }
        }
    }
}
```

Aspetos negativos: ☹

- critério de seleção e ação *hardcoded* na iteração;
- necessário um método novo por cada critério de seleção diferente

2. Parâmetro com critério de seleção

```
interface CheckPerson { boolean test(Person p); }
class CheckMale_18_25 implements CheckPerson {
   public boolean test(Person p) {
      return p.getGender() == Person.Gender.MALE && p.getAge() >= 18 && p.getAge() <= 25;
}  }
public class PersonDemo {
   private static void printWithCheckPerson(List<Person> roster, CheckPerson tester) {
      for (Person p : roster) {
         if (tester.test(p)) { p.printPerson(); }
   }  }
   public static void printWithLocalClassDemo(List<Person> roster) {
      printWithCheckPerson(roster, new CheckMale_18_25());
   }
}
```

Aspeto negativo: para usar um critério de seleção diferente é necessário definir nova classe ☹

Aspeto positivo: o método que apresenta a informação de `Person` não necessita ser alterado função do critério ☺

Critério de seleção passado como argumento do método ☺

3. Parâmetro com classe anónima

```
public class PersonDemo {
   private static void printWithCheckPerson(List<Person> roster, CheckPerson tester) {
      for (Person p : roster) {
         if (tester.test(p)) { p.printPerson(); }
   }  }
   public static void printWithAnonymousClassDemo(List<Person> roster) {
      printWithCheckPerson(
            roster,
            new CheckPerson() {
               public boolean test(Person p) {
                  return p.getGender() == Person.Gender.MALE && p.getAge() >= 18 && p.getAge() <= 25;
            }  }
      );
} }
```

Critério de seleção com classe anónima.
Aspeto positivo: não precisa definir classe ☺

Mais simpática que a versão anterior.
Aspeto negativo: as mesma classe não pode ser usado em diferente sítios. ☹

4. Parâmetro com expressão *lambda*

```
public class PersonDemo {
   private static void printWithCheckPerson(List<Person> roster, CheckPerson tester) {
      for (Person p : roster) {
         if (tester.test(p)) { p.printPerson(); }
   }  }
   public static void printWithLambdaDemo(List<Person> roster) {
      printWithCheckPerson(
            roster,
            (Person p) -> p.getGender() == Person.Gender.MALE && p.getAge() >= 18 && p.getAge() <= 25
      );
}  }
```

O tipo `Person` pode ser omitido se o compilador conseguir inferir (é o caso) ☺

Equivalente à classe anónima mas de escrita mais simples e, consequentemente, de leitura, igualmente, mais simples ☺

5. Parâmetro com `Predicate`

```
public interface Predicate<T> { boolean test(T t); }

public class PersonDemo {
   private static void printWithPredicate(List<Person> roster, Predicate<Person> tester) {
      for (Person p : roster) {
         if (tester.test(p)) { p.printPerson(); }
   }  }
   public static void printPredicateDemo(List<Person> roster) {
      printWithPredicate(
            roster,
            (p) -> p.getGender() == Person.Gender.MALE && p.getAge() >= 18 && p.getAge() <= 25
      );
}  }
```

Não é preciso definir a interface `CheckPerson` porque existe a interface genérica `Predicate`, definida no *package* `java.util.function`, que é equivalente. Serve para filtrar o parâmetro do tipo T ☺

O tipo `Person` pode ser omitido se o compilador conseguir inferir (é o caso) ☺

Expressão *lambda* a ser executada no contexto do predicado

6. Parâmetro com `Consumer`

```
public interface Consumer<T> { void accept(T t); }
public class PersonDemo {
   private static void process(List<Person> roster, Predicate<Person> tester,
                               Consumer<Person> block) {
      for (Person p : roster) {
         if (tester.test(p)) { block.accept(p); }
   }  }
   public static void processPredicateConsumerDemo(List<Person> roster) {
      process(
            roster,
            p -> p.getGender() == Person.Gender.MALE && p.getAge() >= 18 && p.getAge() <= 25,
            p -> p.printPerson()
      );
} }
```

Interface definida no *package* `java.util.function`. Serve para realizar uma ação sobre o parâmetro do tipo T

O mesmo método, não só é genérico no ☺ critério de seleção (`test`), mas também na ação que realiza sobre `Person` (`accept`)

Os parêntesis podem ser omitidos se a ☺ expressão *lambda* for constituída por um único parâmetro (é o caso)

Quer o critério de seleção, quer a ação a ser realizada sobre cada `Person` selecionada, são especificados com expressões *lambda*

7. Parâmetro com `Function`

```
public interface Function<T,R> { R apply(T t); }
public class PersonDemo {
   private static void processWithFunction(List<Person> roster,
            Predicate<Person> tester, Function<Person, String> mapper, Consumer<String> block) {
      for (Person p : roster) {
         if (tester.test(p)) { String data = mapper.apply(p); block.accept(data); }
   }  }
   public static void processWithFunctionDemo(List<Person> roster) {
      processWithFunction(
            roster,
            p -> p.getGender() == Person.Gender.MALE && p.getAge() >= 18 && p.getAge() <= 25,
            p -> p.getEmailAddress(),
            email -> System.out.println(email)
      );
}  }
```

Interface definida no *package* `java.util.function`. Serve para transformar o parâmetro do tipo T no resultado do tipo R

Sobre cada objeto `Person` selecionado, seleciona-se o endereço de email para ser apresentado

8. Método genérico

```
public class PersonDemo {
   private static <X,Y> void processGenerics(
         Iterable<X> source, Predicate<X> tester,
         Function<X, Y> mapper, Consumer<Y> block) {
      for (X p : source) {
         if (tester.test(p)) { Y data = mapper.apply(p);  block.accept(data); }
   }  }
   public static void processGenericsDemo(List<Person> roster) {
      processGenerics(
            roster,
            p -> p.getGender() == Person.Gender.MALE && p.getAge() >= 18 && p.getAge() <= 25,
            p -> p.getEmailAddress(),
            email -> System.out.println(email)
      );
}  }
```

O código do método `processGenerics` ☺ processa qualquer tipo de dados. Para além dos parâmetros tipo (X e Y), o primeiro parâmetro foi alterado de `List` para `Iterable`. Regra geral, todas as coleções são iteráveis

Tudo igual à versão anterior com a exceção do nome do método que é chamado

9. Referências para métodos

```
public class PersonDemo {
   private static <X,Y> void processGenerics(
         Iterable<X> source, Predicate<X> tester,
         Function<X, Y> mapper, Consumer<Y> block) {
      for (X p : source) {
         if (tester.test(p)) { Y data = mapper.apply(p);  block.accept(data); }
   }  }
   public static void processGenericsDemo(List<Person> roster) {
      processGenerics(
            roster,
            p -> p.getGender() == Person.Sex.MALE && p.getAge() >= 18 && p.getAge() <= 25,
            Person::getEmailAddress,
            System.out::println
      );
}  }
```

Pode usar-se uma *referência para método* quando a expressão *lambda* é constituída apenas pela chamada de um método não abstrato ☺

10. Operações agregadas (baseada em *streams*)

```
public class PersonDemo {
    public static void processAggregateOperationsDemo(List<Person> roster) {
        roster
            .stream()
            .filter(p -> p.getGender() == Person.Gender.MALE && p.getAge() >= 18 && p.getAge() <= 25)
            .map(Person::getEmailAddress)
            .forEach(System.out::println);
    }
}
```

`filter`, `map` e `forEach` são operações realizadas sobre *streams*

`filter` e `map` retornam elas próprias *streams*:

- `filter` → *stream* com os elementos que passaram no filtro
- `map` → *stream* com o resultado das transformações

Ação	Operação agregada
Obter sequência de objetos	`Stream<E> stream()`
Filtrar os objetos de acordo com o predicado	`Stream<T> filter(Predicate<? super T> predicate)`
Mapear os objetos filtrados noutro valor	`<R> Stream<R> map(Function<? super T,? extends R> mapper)`
Realizar uma ação para cada elemento mapeado	`void forEach(Consumer<? super T> action)`

Resumo

- Caso de estudo: apresentar, com base num critério de seleção, informação de pessoas. Os objetos do tipo `Person` estão presentes numa coleção

- Expressões *lambda*

- Referência a métodos

O que é?

- Forma simplificada de escrever a definição de uma classe anónima
- Uma expressão *lambda* é identificada pelo operador seta (->)
- O tipo interface ou classe que a expressão *lambda* está a definir não pode conter mais do que um método abstrato
  - Métodos estáticos não condicionam
  - No caso de interfaces, métodos *default* (com implementações por omissão) também não condicionam

Sintaxe

(parâmetros formais) -> expressão ou bloco de instruções

- Exemplos:

```
(Person p) -> p.getAge() >= 18 && p.getAge() <= 25

(Person p, int l, int h) -> p.getAge() >= l && p.getAge() <= h

(p, l, h) -> p.getAge() >= l && p.getAge() <= h

p -> p.getAge() >= 18 && p.getAge() <= 25

p -> { int age = p.getAge(); return age >= 18 && age <= 25; }

_ -> System.out.println(“Xpto”);
```

Sintaxe – parâmetros formais

(*parâmetros formais*) -> *expressão ou bloco de instruções*

```
(Person p) -> p.getAge() >= 18 && p.getAge() <= 25
```

- Parâmetros formais:
  - Declarados dentro de parênteses curvos
  - Conjunto de parâmetros formais que a função *lambda* recebe separados por vírgula (‘,’)

    ```
    (Person p, int l, int h) -> p.getAge() >= l && p.getAge() <= h
    ```
  - Pode omitir-se o tipo de dados dos parâmetros formais (o compilador consegue inferir os tipos de dados)

    ```
    (p, l, h) -> p.getAge() >= l && p.getAge() <= h
    ```
  - Os parênteses curvos podem ser omitidos se existir apenas 1 parâmetro

    ```
    p -> p.getAge() >= 18 && p.getAge() <= 25
    ```
  - Os parâmetros formais pode ser omitidos por completo se a expressão *lambda* não envolver referências aos parâmetros

    ```
    _ -> System.out.println(“Xpto”);
    ```
  - Usa-se apenas *abrir e fechar parêntesis* se a função for constituída por 0 parâmetros

    ```
    () -> <expressão>
    ```

Sintaxe – expressão ou bloco de instruções

(*parâmetros formais*) -> *expressão ou bloco de instruções*

```
p -> p.getAge() >= 18 && p.getAge() <= 25
```

- Expressão ou bloco de instruções. Cenários:
  - Método onde se usa a expressão *lambda* retorna um valor:
    - Se a expressão for constituída por uma única expressão, deve escrever-se apenas a expressão
    - Pode usar-se a cláusula `return` e, nesse caso, tem de se envolver a expressão entre chavetas

```
p -> { return p.getAge() >= 18 && p.getAge() <= 25; }
```

  - Método onde se usa a expressão *lambda* não retorna qualquer valor (método `void`)
    - Se a expressão for constituída por uma única instrução, pode omitir-se as chavetas

```
email -> System.out.println(email)
```

  - Expressão *lambda* constituída por um bloco de instruções
    - O bloco de instruções tem de ser envolvido por chavetas

```
p -> { int age = p.getAge(); return age >= 18 && age <= 25; }
```

Exemplo com expressões *lambda*

```
public class Calculator {

   interface IntegerMath { int operation(int a, int b); }
   public int operateBinary(int a, int b, IntegerMath op) { return op.operation(a, b); }

   public static void main(String... args) {

      Calculator myApp = new Calculator();
      IntegerMath addition = (x, y) -> x + y;
      IntegerMath subtraction = (x, y) -> x - y;
      System.out.println("40 + 2 = " +  myApp.operateBinary(40, 2, addition));
      System.out.println("20 - 10 = " +  myApp.operateBinary(20, 10, subtraction));
   }
}
```

Acesso a variáveis do contexto envolvente

- O código de uma expressão *lambda* tem acesso às variáveis definidas no contexto envolvente, tal como nas classes locais e anónimas
  - Exigidas as mesmas regras no acesso a variáveis locais, ou seja, as variáveis locais têm de ser finais ou *efetivamente finais* (ver slides que trata classes aninhadas)
  - Ao contrário das classes locais e anónimas, as expressões *lambda* não apresentam problemas de sombreamento dos nomes de variáveis locais
    - O código da expressão *lambda* é considerado estar no scope do método onde a expressão *lambda* é usada (*lexically scoped*), ou seja, uma expressão *lambda* não introduz um novo nível de contexto

Exemplo com variáveis do contexto envolvente

```
public class LambdaScopeTest {
    public int x = 0;
    class FirstLevel {
        public int x = 1;
        void methodInFirstLevel(int x) {
            int z = 2;
            Consumer<Integer> myConsumer = (y) ->
            {
                System.out.println("x = " + x);
                System.out.println("y = " + y);
                System.out.println("z = " + z);
                System.out.println("this.x = " + this.x);
                System.out.println(
                        "LambdaScopeTest.this.x = " +
                        LambdaScopeTest.this.x);
            };
            myConsumer.accept(x);
    } }
```

```
    public static void main(String... args) {
        LambdaScopeTest st = new LambdaScopeTest();
        LambdaScopeTest.FirstLevel fl =
                st.new FirstLevel();
        fl.methodInFirstLevel(23);
    }
}
```

Qualquer definição da variável x na expressão *lambda* produz erro de compilação (porque já existe uma definição de x no mesmo *scope*)

```
x = 23
y = 23
z = 2
this.x = 1
LambdaScopeTest.this.x = 0
```

Tipo de uma expressão *lambda*

- Como determinar o tipo de uma expressão *lambda*?
  - Depende do tipo de interface ou classe que a expressão *lambda* implementa
- Por exemplo: a mesma expressão *lambda*

```
p -> p.getGender() == Person.Sex.Male && p.getAge() >= 18 && p.getAge() <= 25
```

pode ser de diferentes tipos

  - Se for usada na chamada do método

```
private static void printPersonsWithCheckPerson(List<Person> roster,
                                                CheckPerson tester)
```

  a expressão *lambda* é do tipo `CheckPerson`

  - Se for usada na chamada do método

```
private static void printPersonsWithPredicate(List<Person> roster,
                                              Predicate<Person> tester)
```

  a expressão *lambda* é do tipo `Predicate<Person>`

Resumo

- Caso de estudo: apresentar, com base num critério de seleção, informação de pessoas. Os objetos do tipo `Person` estão presentes numa coleção

- Expressões *lambda*

- Referência a métodos

O que é?

- Corresponde a um nome de um método não abstrato

- Quando se deve usar?
  - Quando a expressão *lambda* corresponde à chamada de um único método; nesse caso é preferível usar o nome do método pois simplifica a escrita e a leitura do código

Exemplo (1 de 3)

- Ordenar um *array* de objetos do tipo `Person` pela idade, usando três variantes:
  1. Expressão *lambda* com interface funcional `Comparator`
  2. Expressão *lambda* com método `compareByAge` definido na classe `Person`
  3. Referência para método `compareByAge`

```
public class MethodReferencesDemo {
   public static void main(String... args) {
      // ... Iniciação de personAsArray: Person[] rosterAsArray
      // With lambda expression comparing birthdays explicitly
      Arrays.sort( rosterAsArray, (a, b) -> a.getBirthday().compareTo(b.getBirthday()) );
   }
}
 // Assinatura do método sort: static <T> void sort(T[] a, Comparator<? super T> c)
```

Exemplo (2 de 3)

- Ordenar um *array* de objetos do tipo `Person` pela idade, usando três variantes:
  1. Expressão *lambda* com interface funcional `Comparator`
  2. Expressão *lambda* com método `compareByAge` definido na classe `Person`
  3. Referência para método `compareByAge`

```
public class MethodReferencesDemo {
   public static void main(String... args) {
      // ... Iniciação de personAsArray: Person[] rosterAsArray
      // With lambda expression calling Person.compareByAge method
      Arrays.sort( rosterAsArray, (a, b) -> Person.compareByAge(a, b) );
   }
}
 // Assinatura do método sort: static <T> void sort(T[] a, Comparator<? super T> c)
```

Exemplo (3 de 3)

- Ordenar um *array* de objetos do tipo `Person` pela idade, usando três variantes:
  1. Expressão *lambda* com interface funcional `Comparator`
  2. Expressão *lambda* com método `compareByAge` definido na classe `Person`
  3. Referência para método `compareByAge`

```
public class MethodReferencesDemo {
   public static void main(String... args) {
      // ... Iniciação de personAsArray: Person[] rosterAsArray
      // With lambda expression with method reference
      Arrays.sort(rosterAsArray, Person::compareByAge);
   }
}
 // Assinatura do método sort: static <T> void sort(T[] a, Comparator<? super T> c)
```

Tipos de referências para métodos

Tipo	Sintaxe	Exemplos
Referência para um método estático	*ContainingClasse*::*staticMethodName*	`Person::compareByAge` `MethodReferencesExample::appendStrings`
Referência para um método de instância de um determinado objeto	*containingObject*::*instanceMethodName*	`myComparisonProvider::compareByName` `myApp::appendStrings2`
Referência para um método de instância de um determinado tipo	*ContainingType*::*methodName*	`String::compareToIgnoreCase` `String::concat`
Referência para um construtor	*ClassName*::*new*	`HashSet::new`

Exemplo – referência para método estático

```
public class StaticMethodReferenceDemo {
   public static <T> T mergeThings(T a, T b, BiFunction<T, T, T> merger) { return merger.apply(a, b); }

   public static String appendStrings(String a, String b) { return a + b; }

   public static void main(String[] args) {
      // Calling the method mergeThings with a lambda expression
      System.out.println(StaticMethodReferenceDemo.mergeThings("Hello ", "World!", (a, b) -> a + b));
      // Reference to a static method
      System.out.println(StaticMethodReferenceDemo.mergeThings("Hello ", "World!",
            StaticMethodReferenceDemo::appendStrings));
   }
}
```

Exemplo – referência para método de instância de um determinado objeto

```
public class ObjectInstanceMethodReferenceDemo {
   public static <T> T mergeThings(T a, T b, BiFunction<T, T, T> merger) { return merger.apply(a, b); }

   public String appendStrings2(String a, String b) { return a + b; }

   public static void main(String[] args) {
      ObjectInstanceMethodReferenceDemo myApp = new ObjectInstanceMethodReferenceDemo();

      // Reference to an instance method of a particular object
      System.out.println(ObjectInstanceMethodReferenceDemo.mergeThings("Hello ", "World!",
            myApp::appendStrings2));
   }
}
```

Exemplo – referência para método de instância de um objeto de um determinado tipo

```
public class TypeInstanceMethodReferenceDemo {
   public static <T> T mergeThings(T a, T b, BiFunction<T, T, T> merger) { return merger.apply(a, b); }

   public static void main(String[] args) {
      TypeInstanceMethodReferenceDemo myApp = new TypeInstanceMethodReferenceDemo();

      // Reference to an instance method of an arbitrary object of a particular type
      System.out.println(TypeInstanceMethodReferenceDemo.mergeThings("Hello ", "World!",
            String::concat));
   }
}
```